# Machine Learning Project

## Milestone 1 Report

### Online Games Popularity Prediction

### Team #19

*Preprocessing • Feature Selection • Regression Modeling*

**Team Members:**

1. ***Raafat Mahmoud Abd El-Azim Marzouk*** ***2023170211***
2. ***Khattab Abd El-Moniem Khattab Mohamed*** ***2023170204***
3. ***Ahmed Eid Awad Ali*** ***2023170048***
4. ***Abd El-Rahman Mosa Abd El-Hakam Mosa*** ***2023170347***
5. ***Zainab Alaa Ali Ibrahim*** ***2023170249***
6. ***Youssef El-Sayed Mohamed Ahmed*** ***2023170715***

## 1. Introduction

The goal of this project is to predict how popular an online game will be on the Steam platform. Popularity is quantified by the RecommendationCount — the number of user recommendations a game receives. Understanding what drives game popularity can help developers and publishers make better decisions at design and marketing stages.

The dataset contains rich metadata about Steam games: genre flags, platform availability, pricing, achievement counts, SteamSpy player estimates, release dates, and more. Milestone 1 covers three main phases: (1) full preprocessing of all features, (2) feature selection and analysis, and (3) training and evaluating at least two regression models with significant differences.

## 2. Dataset Overview

The dataset consists of two source files: train_data.csv, which contains game metadata and the target column RecommendationCount, and idlist.csv, which maps QueryID to GameName. After merging on the QueryID/ID key, the combined dataset has approximately 13,000+ rows and more than 70 columns spanning numeric, boolean, text, and date-type features.

Key feature categories include:

- Popularity metrics: SteamSpyOwners, SteamSpyPlayersEstimate and their variance columns.
- Game content: AchievementCount, AchievementHighlightedCount, ScreenshotCount, MovieCount, DLCCount.
- Pricing: PriceInitial, PriceFinal, IsFree, GenreIsFreeToPlay.
- Genre flags: GenreIsIndie, GenreIsAction, GenreIsAdventure, GenreIsRPG, and 10+ others.
- Platform flags: PlatformWindows, PlatformLinux, PlatformMac.
- Category flags: CategoryMultiplayer, CategoryCoop, CategoryMMO, CategorySinglePlayer, etc.
- Temporal: ReleaseDate (parsed into ReleaseYear and ReleaseMonth).
- Text / URL columns (dropped): ShortDescrip, AboutText, HeaderImage, Website, etc.

## 3. Preprocessing

All features were preprocessed before any feature selection or modeling, as required. The steps below were applied in order.

### 3.1 Dropping Irrelevant Columns

Free-text fields (ShortDescrip, AboutText, DetailedDescrip, Reviews, LegalNotice, SupportedLanguages, Website, HeaderImage, DRMNotice, ExtUserAcctNotice), URL columns, and identifier columns (QueryID, ResponseID, GameName, ID) were dropped. These carry no numeric regression signal; processing raw text would require NLP pipelines beyond the scope of this milestone. The PriceCurrency column was also dropped because it was a constant value (USD) across all rows, contributing zero variance.

A total of 25 columns were dropped in this step.

### 3.2 Date Parsing

The ReleaseDate column was parsed using pandas to_datetime() with errors="coerce" to handle malformed entries. Two new numerical features were extracted: ReleaseYear and ReleaseMonth. ReleaseYear captures generational trends in the gaming market (older games vs. modern ones), while ReleaseMonth captures seasonal release patterns (e.g., holiday launches). The original ReleaseDate column was then dropped.

### 3.3 Boolean Encoding

All 35 boolean columns (genre flags, platform flags, category flags, and others) were standardised to integer values of 0 and 1. The mapping handled multiple formats found in the raw data: Python True/False, string "TRUE"/"FALSE", and numeric 1/0. Missing values in boolean columns were filled with 0, representing the absence of the characteristic.

### 3.4 Median Imputation for Numeric Columns

Seventeen numeric columns (SteamSpy estimates, achievement counts, prices, screenshot counts, etc.) were imputed using the median strategy via sklearn SimpleImputer. The median was preferred over the mean because these columns exhibit heavy right-skew — the median is robust to extreme outliers and avoids inflating imputed values.

### 3.5 IQR Outlier Capping

Extreme outliers in the numeric features were capped using the Interquartile Range (IQR) method. For each numeric column, values below Q1 - 1.5*IQR were clipped to the lower fence, and values above Q3 + 1.5*IQR were clipped to the upper fence. This prevents extreme SteamSpy estimate outliers from distorting regression slope estimates while preserving the overall distribution.

### 3.6 Log1p Transformation

Five columns — SteamSpyOwners, SteamSpyOwnersVariance, SteamSpyPlayersEstimate, SteamSpyPlayersVariance, and the target

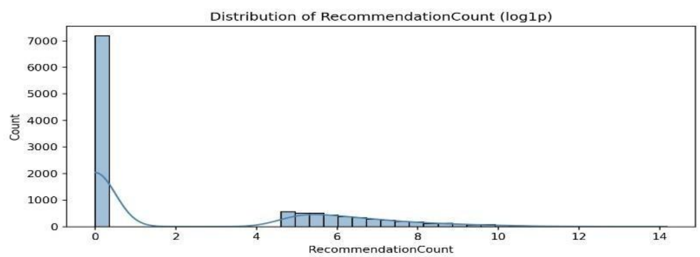
RecommendationCount — were log1p-transformed (i.e., log(1 + x)). These columns span many orders of magnitude (from zero to millions), which violates the linearity and homoscedasticity assumptions of linear regression. Log compression brings these into a manageable range and improves model performance. Using log1p instead of log ensures safe handling of zero values.

Important: all predictions from regression models are in log-space. To convert predictions back to raw recommendation counts, the inverse transformation np.expm1() must be applied.

# 4. Feature Analysis

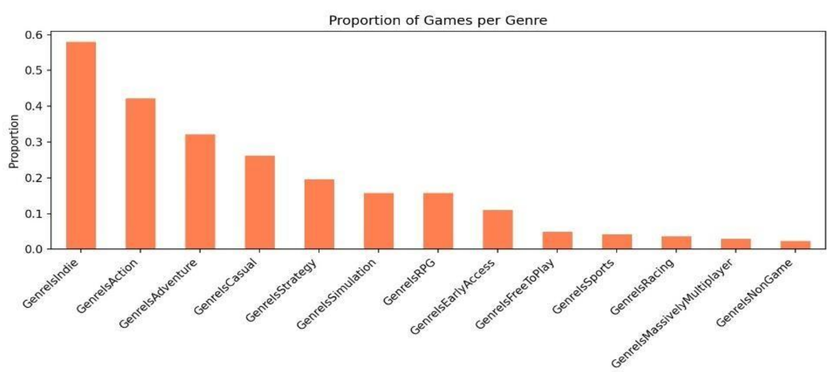
## 4.1 Target Distribution

Figure 1 shows the distribution of RecommendationCount after log1p transformation. Before transformation the distribution was extremely right-skewed with the vast majority of games having zero or near-zero recommendations. After transformation, two visible clusters appear: one at log1p ≈ 0 (games with very few recommendations) and one centred around log1p ≈ 5–6 (games with a moderate-to-high number of recommendations). This bimodal pattern suggests a meaningful separation between obscure and popular games in the dataset.




## 4.2 Genre Distribution

Figure 2 shows the proportion of games belonging to each genre. Indie games dominate (~58%), followed by Action (~42%) and Adventure (~32%). Many games belong to multiple genres simultaneously. The low-proportion genres (Sports, Racing, MassivelyMultiplayer) are present in less than 5% of the dataset, which limits their predictive power.




## 4.3 Correlation Analysis

Figure 3 presents the correlation matrix for the top 15 features plus the target. Several key patterns emerge:

- SteamSpy cluster (r ≈ 0.51–0.61 with target): SteamSpyPlayersEstimate, SteamSpyOwners, SteamSpyPlayersVariance, and SteamSpyOwnersVariance are all strongly inter-correlated (r > 0.95) and show the highest correlation with RecommendationCount.

- ReleaseYear (r = -0.40 with target): Newer games tend to have fewer recommendations, possibly because older popular games have had more time to accumulate recommendations.
- AchievementCount (r = 0.36): Games with more achievements tend to receive more recommendations, possibly reflecting higher production quality and engagement.
- Price (r ≈ 0.30): Higher-priced games correlate slightly more with higher recommendation counts, though free-to-play games (GenreIsFreeToPlay) show a negative effect on price.
- CategoryMultiplayer (r = 0.21): Multiplayer games receive somewhat more recommendations, likely reflecting the social nature of multiplayer that encourages sharing opinions.

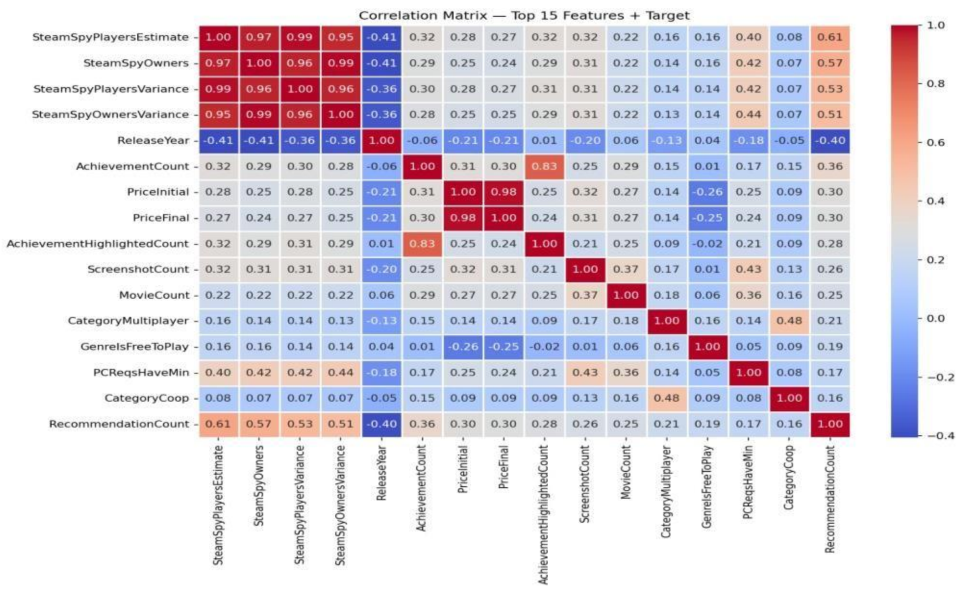



*Figure 3: Correlation Matrix — Top 15 Features + Target (RecommendationCount)*

# 5. Feature Selection

Feature selection was performed using sklearn SelectKBest with the f_regression scoring function, which measures the linear statistical relationship between each feature and the target using an F-test. The top 25 features by F-score are shown in Figure 4, and the top K = 20 features were selected for model training.

The four SteamSpy columns (PlayersEstimate, Owners, PlayersVariance, OwnersVariance) dominate the rankings by a wide margin, with F-scores of 4,000–6,500. ReleaseYear comes next at ~2,100, followed by AchievementCount at ~1,700. Price columns, screenshot/movie counts, multiplayer flags, and system requirements round out the selected features. Genre flags such as GenreIsFreeToPlay and GenreIsCasual were included due to meaningful Fscores, while low-signal features like GenreIsRPG and PCReqsHaveRec were near the boundary.

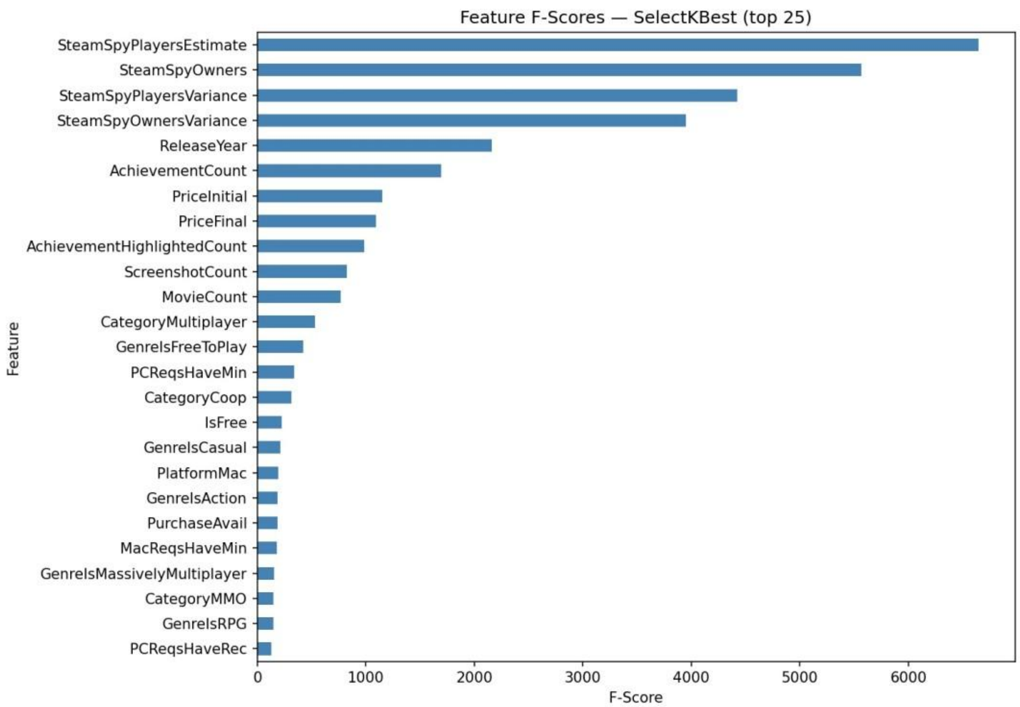



*Figure 4: Feature F-Scores via SelectKBest (top 25 shown, top 20 selected)*

Features dropped after preprocessing and selection (with justification):

- Text/URL/ID columns: zero numeric regression signal; would require NLP.
- PriceCurrency: constant value (all USD), zero variance.
- ReleaseMonth: very low F-score; monthly seasonality weak vs. year.
- Low-ranking binary flags: CategoryIncludeSrcSDK, CategoryIncludeLevelEditor, CategoryVRSupport, etc. — F-scores below threshold after selection.

## 6. Regression Models

Four regression models were trained and evaluated. All models predict RecommendationCount in log1p space. The dataset was split into Training (70%), Validation (15%), and Test (15%) sets using stratified random sampling with random_state=42 for reproducibility. StandardScaler was applied to normalise features for the linear models. **6.1 Data Splits**

| Split | Proportion | Approx. Rows |
|---|---|---|
| Training | 70% | ~7,900 |
| Validation | 15% | ~1,700 |
| Test | 15% | ~1,700 |

### 6.2 Model 1: Linear Regression

Linear Regression fits a hyperplane by minimising the sum of squared residuals (OLS). It assumes a linear relationship between features and the target, independence of errors, and homoscedasticity. It serves as the simplest possible baseline and provides interpretable coefficients showing the direction and magnitude of each feature's effect.

Test set performance: MAE = 1.427, RMSE = 1.940, $R^2 = 0.644$.

The model explains 64.4% of variance in log1p(RecommendationCount). The RMSE of 1.94 in log-space translates to meaningful uncertainty in real-world recommendation counts. The model struggles with the bimodal target distribution, particularly with the zerorecommendation cluster.

### 6.3 Model 2: Ridge Regression

Ridge Regression extends OLS by adding an L2 regularisation penalty ($\alpha$=10) to the loss function: Loss = RSS + $\alpha * \sum\beta^2$. This shrinks the magnitude of coefficients, particularly for correlated features. Given that several SteamSpy columns are near-perfectly correlated ($r > 0.95$), Ridge is appropriate — it prevents any single correlated feature from dominating the model and reduces overfitting.

Test set performance: MAE = 1.443, RMSE = 1.949, $R^2 = 0.641$.

Performance is nearly identical to plain Linear Regression on this dataset, which indicates that L2 regularisation alone does not resolve the key challenge (the bimodal distribution and non-linear patterns). The regression line in Figure 7 shows the fit has a slope of $y = 0.63x + 0.83$, indicating systematic under-prediction at high actual values, a known weakness of linear models on skewed targets.

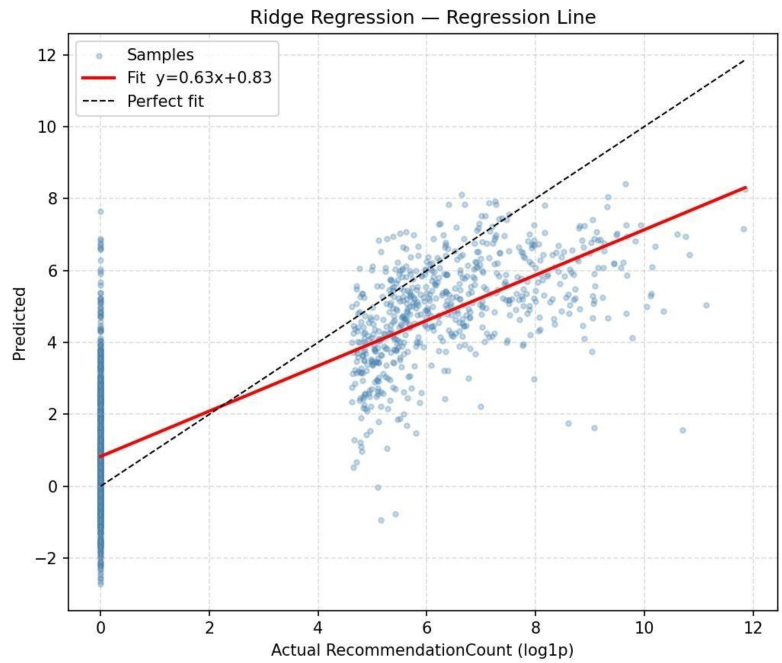



*Figure 5: Ridge Regression — Fitted Line vs. Perfect Fit*

### 6.4 Model 3: Random Forest Regressor

Random Forest is an ensemble of 200 decision trees (max_depth=12, min_samples_leaf=5). Each tree is trained on a bootstrap sample with a random subset of features at each split. Predictions are averaged across all trees. Key differences from the linear models: (1) no linearity assumption, (2) automatically captures non-linear feature interactions, (3) robust to outliers, and (4) requires no feature scaling. These properties make it significantly better suited to this dataset.

Test set performance: MAE = 0.814, RMSE = 1.520, $R^2$ = 0.782.

A major improvement over the linear models. $R^2$ = 0.782 means the model explains 78.2% of variance. The RMSE drops from ~1.94 to 1.52 — a 22% reduction. The Actual vs. Predicted scatter (Figure 6) shows predictions much closer to the perfect-fit line.

### 6.5 Model 4: Gradient Boosting Regressor

Gradient Boosting builds 300 shallow trees sequentially (learning_rate=0.05, max_depth=5, subsample=0.8). Each new tree focuses on correcting the residuals of all previous trees by fitting the negative gradient of the loss function. Key differences from Random Forest: (1) sequential rather than parallel training, (2) corrects previous errors explicitly, (3) subsampling reduces variance, (4) slower to train but often achieves higher accuracy.

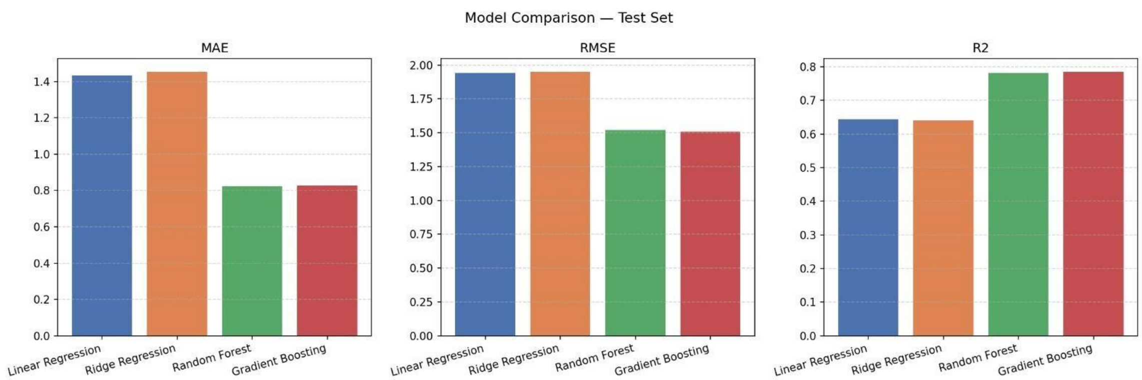
Test set performance: MAE = 0.814, RMSE = 1.510, $R^2$ = 0.785.

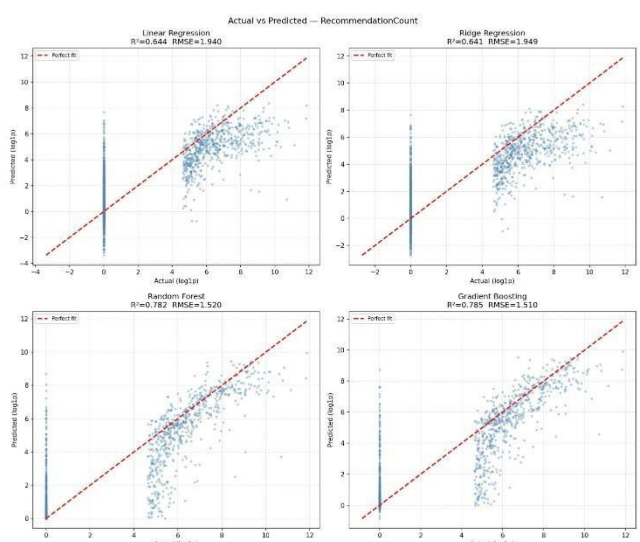
The best-performing model overall. Marginally outperforms Random Forest with $R^2$ = 0.785 and RMSE = 1.510. Both tree-based ensembles significantly outperform the linear models.

# 7. Model Comparison

Table 2 summarises all four models on the held-out test set (all values in log1p space).

| Model | MAE | RMSE | $R^2$ | Rank |
|---|---|---|---|---|
| Linear Regression | 1.427 | 1.940 | 0.644 | 3rd |
| Ridge Regression (α=10) | 1.443 | 1.949 | 0.641 | 4th |
| Random Forest | 0.814 | 1.520 | 0.782 | 2nd |
| Gradient Boosting | 0.814 | 1.510 | 0.785 | 1st ★ |




The gap between linear models ($R^2 \approx 0.64$) and tree-based models ($R^2 \approx 0.78$) is substantial — approximately 14 percentage points in explained variance. This confirms that the relationship between game features and RecommendationCount is highly non-linear. Gradient Boosting edges out Random Forest in both RMSE and $R^2$ and is the recommended model.

## 8. Further Techniques Applied

### 8.1 Target Variable Transformation

Applying log1p to RecommendationCount before training was the single most impactful preprocessing step. The raw target ranged from 0 to hundreds of thousands, violating normality assumptions. After transformation, the scale was compressed to 0–14, reducing the influence of extreme outliers on all model loss functions.

### 8.2 StandardScaler for Linear Models

Feature standardisation (zero mean, unit variance) was applied to Linear and Ridge Regression inputs. This ensures that coefficients are on a comparable scale and that Ridge's L2 penalty is applied consistently across all features. Tree-based models were trained on the raw (unscaled) selected features since decision trees are scale-invariant.

### 8.3 Validation Set Strategy

A three-way 70/15/15 split was used. The validation set allowed monitoring of model generalisation during development (e.g., confirming that Ridge's alpha=10 did not overregularise) without contaminating the final test set evaluation.

## 9. Conclusion

This milestone successfully completed all three required tasks: full preprocessing of all dataset features, feature analysis and selection, and training of four regression models with clear differences in methodology and performance.

Our initial intuition was that SteamSpy-based player metrics (Owners, PlayersEstimate) would be the most powerful predictors of recommendation count, since popular games are both widely owned and widely recommended. This was confirmed strongly by the correlation analysis and F-score ranking.

We also hypothesised that simple linear models would struggle given the heavy skewness and likely non-linear interactions between features. This was proven correct: linear and Ridge regression plateaued at $R^2 \approx 0.64$, while the tree-based models reached $R^2 \approx 0.78$.

One unexpected finding was the strength of ReleaseYear as a negative predictor ($r = -0.40$). We had expected newer games to have more recommendations due to a larger active player base, but the data shows the opposite — likely because older successful games have accumulated recommendations over many years.

Going into Milestone 2, the recommended model is Gradient Boosting. Potential improvements include hyperparameter tuning (via GridSearchCV), feature engineering from text fields, and experimenting with XGBoost or LightGBM for additional performance gains.